Simulation Orchestration & Async Integration

Technical Progress Report (Week 3)

Pranjal Kumar Shukla

FOSSEE Internship – Progress Report

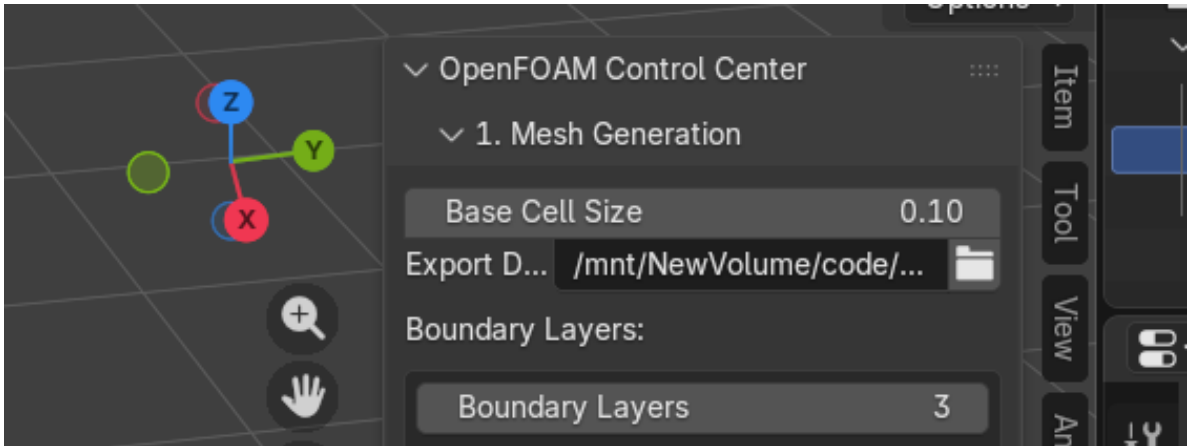
SCAI School

VIT Bhopal University

March 21, 2026

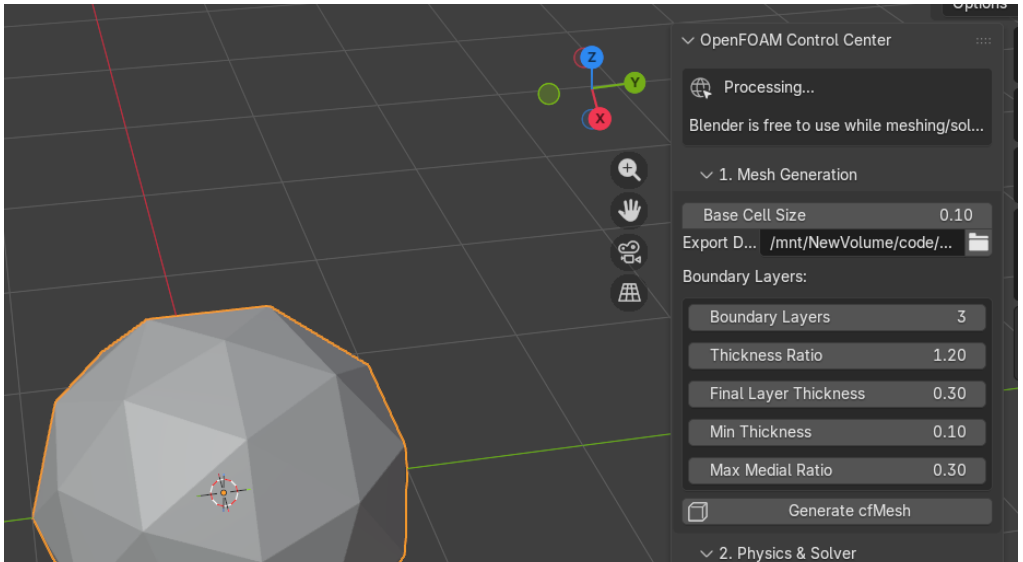
1. Multi-Solver Integration

- **Objective**: Evolve the addon from a pure meshing utility into a complete CFD orchestrator.
- **Implementation**: Integrated GUI controls to configure real-world physical bounds.
  - Selection between `icoFoam` (Transient) and `simpleFoam` (Steady-State).
  - Parametric inputs for Kinematic Viscosity (ν) and Inlet Velocity Vectors.
- **Dynamic Dictionary Gen**: The backend Python engine cleanly writes the `0/U`, `0/p`, and `system/controlDict` configurations natively across OS boundaries.



2. Asynchronous Execution Pipeline

- **The Bottleneck**: OpenFOAM solver iterations freeze Blender's main UI thread, forcing users to "blindly wait."
- **The Solution**:
  - Implemented Python `threading.Thread` for all terminal background processes.
  - Registered a recurring `bpy.app.timers` hook to constantly poll Global State variables.
- **Result**: The Blender viewport remains 100% fluid and functional while `snappyHexMesh` or `simpleFoam` chew through CPU cycles.



3. Resolving Python Architecture Bugs

Developing highly compartmentalized Blender extensions led to severe edge cases:

- `bl_info` **Registration**: A typo in the registration header (`maxbl_info`) caused the entire Addon to become invisible to Blender's plugin parser.
- **Module NotFound Errors**: Attempting standard Python `import run_cfmesh` failed catastrophically inside Blender's complex system path.
- **The Fix**: Implemented strict **Relative Imports** (`from . import run_cfmesh`) forcing Blender to reliably resolve the module within its own localized ZIP container.

4. Navigating OpenFOAM 11's New Paradigm

The "simpleFoam is Superseded" Error

OpenFOAM 11 entirely deleted standard standalone solver aliases in favor of a universal module execution layer.

- **The Migration**: We transitioned the native Blender call from `simpleFoam` to `foamRun -solver incompressibleFluid`.
- **Dictionary Overhauls**:
  - OF-11 dynamically determines if a simulation is Steady vs Transient based on the `ddtSchemes` variable.
  - Implemented logic to inject `steadyState` and `SIMPLE` equation blocks dynamically depending on the user's dropdown choice in the Blender UI.
  - Added explicit generation for `constant/momentumTransport` to satisfy OF-11's modular turbulence checking.

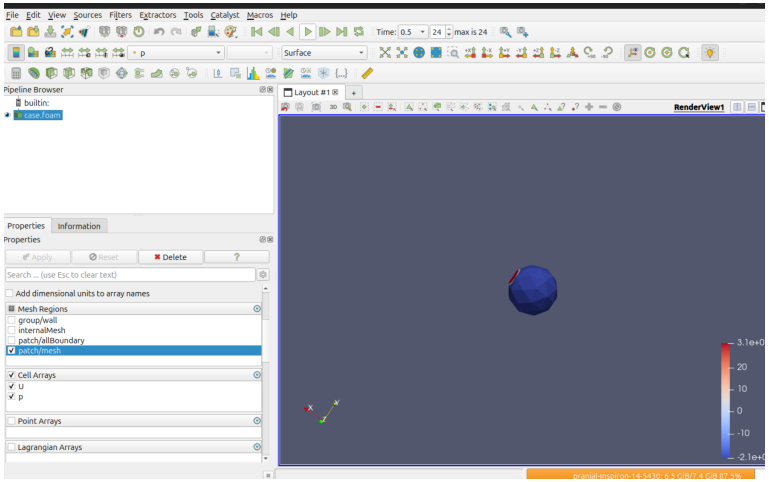
5. Fixing the Hollow Mesh Failure

- **Issue**: `snappyHexMesh` frequently returned an un-meshed solid cube instead of the desired object.
- **Cause #1**: The `locationInMesh` point was hardcoded to `(0,0,0)`. If an object sits at the exact origin, OpenFOAM attempts to mesh the *inside* of it instead of the surrounding wind tunnel!
- **Cause #2**: User-favorites like the "Suzanne" monkey are not watertight surfaces.
- **Resolution**: Shifted the internal meshing coordinate to `(1.9, 1.9, 1.9)` to guarantee location in the external fluid domain.

6. ParaView Integration

The ultimate goal of Phase 3 was seamless End-to-End orchestration.

- A new native UI button generates the '.foam' environment trigger file and launches ParaView automatically.
- Fully bridges the gap between Blender's 3D workspace, OpenFOAM's simulation engine, and ParaView's post-processing rendering array.



*Steady-State Pressure visualized in ParaView

7. Upcoming weeks: Advanced Scenarios

1. **Dynamic Meshing**: Implementing parameter bounds for moving objects.
2. **In-Blender Post Processing**: Basic pseudo-color visualization onto Blender surfaces without needing ParaView.
3. **Turbulence Modeling**: Expanding the solver boundaries to support sophisticated `k-epsilon` and `k-omega` RANS models.

Thank You!

Questions?